Video presentation flow and speaking guide

15 minutes / 5 members / 4 grading criteria

Formal title: **Analista: A Web-Based Thesis Review and Feedback Management System for Students and Volunteer IT Evaluators**.

Suggested duration: **15 minutes**, with approximately **3 minutes per member**. Adjust this rehearsal target if the teacher specifies a different video limit.

The grading criteria are GUI 25%, Algorithm 25%, Visualization 25%, and Database 25%. Each criterion needs an explanation and visible evidence in the video. Equal grading weights do not require identical screen time.

Flow and speaking assignments

Video time	Presenter	Assigned part	Grading evidence
00:00-03:00	Member 1	Introduction, purpose, GUI, and user roles	Home, navigation, evaluator search/filter/profile, and clear role-specific workspaces
03:00-06:00	Member 2	Student submission and review-request algorithm	Project form, PDF upload, evaluator eligibility checks, and pending request
06:00-09:00	Member 3	Evaluator review and student response	Accepted/completed transitions, assigned PDF, recommendations, delivery, and student rating
09:00-12:00	Member 4	Visualization and administration	Ratings average/distribution, review progress, review monitoring, and a brief admin verification/history view
12:00-15:00	Member 5	Database connections and closing	Table relationships, a readable joined query/result, document metadata, and the final workflow recap

Use one primary example throughout: **Campus Appointment and Queue Management System**. Use **By request only** for its main demonstration so the student can request a particular evaluator.

Story: introduction -> student submits a PDF -> student requests review -> evaluator accepts -> evaluator completes feedback -> student receives feedback and rates it -> visualizations reflect the records -> database explains how the records connect.

Member 1 - introduction and GUI

00:00-00:40: introduce the group and purpose. Display the project title and five members' names.

Suggested narration:

> *Good day. We are presenting Analista, a web-based thesis review and feedback management system. We developed it to organize how students connect with volunteer IT evaluators, submit thesis documents, receive recommendations, and follow their review progress.*

No interviews were conducted. Explain the intended purpose and design objectives; user needs still require validation. Do not present assumed student experiences as interview findings.

00:40-02:20: show the interface. Open the home page, Find evaluators, and one approved evaluator profile. Demonstrate a search or expertise filter. Explain the navigation, readable cards, action buttons, and clear labels. Briefly show the layout at a phone-sized viewport if already recorded.

02:20-03:00: explain the roles and introduce the example project. Students manage submissions; approved evaluators provide reviews; administrators review evaluator applications and manage the community.

Handover:

> *We will now follow one student project through the review process.*

Member 2 - student workflow and algorithm

03:00-04:15: show a student submission. Open My projects, create the example project, enter its relevant details, and upload a valid sample PDF. Select By request only. Show the submitted project and its PDF version. Cut or speed up repetitive typing in the edit.

04:15-05:30: explain the review-request decision process. Use a short flowchart overlay and point to the matching application behavior:

1. Check the student's membership and permission to request a review for the project.
2. Check that the project is ready and that the selected PDF belongs to it.
3. Check that the evaluator is approved, active, accepting requests, and within capacity.
4. Prevent conflicts such as an evaluator reviewing their own group's project.
5. Check applicable review-request and project rules, then save a pending request.

The evaluator must still explicitly accept the pending request. Passing the request checks does not automatically start a review.

Show one rehearsed validation case, such as a missing required field, then correct it and continue. If demonstrating a capacity or eligibility refusal, prepare the actual account state beforehand and show the real response.

Suggested narration:

> *Analista uses rule-based checks to decide whether a review request is allowed. It checks the project, document version, evaluator eligibility, available capacity, and conflicts before saving the request.*

05:30-06:00: submit the request and show Pending review. Explain that the request is linked to the chosen PDF version.

Code references for preparation: `ReviewController::store`, `ReviewController::respond`, and `ProjectAccess`. A brief highlighted code excerpt or simple decision flow can support the explanation. The application does not use AI ranking to assign evaluators.

Handover:

> *The request is now pending. Our next presenter will demonstrate the evaluator's response.*

Member 3 - evaluator review and student response

06:00-06:50: accept and open the assigned document. Switch to the prepared evaluator session, open My reviews, and choose Accept & start review. Show that the PDF becomes accessible for the accepted assignment.

06:50-07:50: deliver useful feedback. Show a prepared, substantive summary and, if available, an annotated PDF. Complete and send the review. Explain the status transitions:

```
Pending review -> In review -> Completed
```

These transitions determine the available actions and access permissions. For the primary request-only example, completing a review marks the project completed. Volunteer projects use a different completion rule based on their required reviewers and remaining active reviews.

07:50-09:00: return to the student. Show the notification or message, open the delivered review, and submit a rating and a short comment. A rating is available after completion. This creates the record that Member 4 will show on My ratings.

Suggested narration:

> *The evaluator reviews a specific document version and sends recommendations to the student group. After the review is completed, the students can read the result and rate the review they received.*

Handover:

> *These completed reviews and student ratings also update the information shown in the system's visualizations.*

Member 4 - visualization and administration

09:00-10:00: show My ratings in the evaluator menu. Point out the overall average, total ratings, distribution bars, and individual feedback cards. Apply a star filter and explain that it filters the cards while the overall summary still represents all ratings received by that evaluator.

Explain the calculation:

```
Average rating = sum of received star values / number of received ratings
```

Use the actual values displayed in the recording. For a simple mathematical illustration, ratings of 4 and 5 have an average of 4.5; label this as an illustration unless those are the actual recorded values.

10:00-11:00: show evaluator-slot progress and completed-review counts. Use a prepared volunteer-mode project with two completed distinct evaluators, no other accepted reviews, and four required reviewers. Its bar shows two of four evaluator slots filled. Identify it as a separate prepared example; the main project uses request-only mode. Avoid trying to request a specific evaluator while that project is in volunteer mode.

```
Slot-fill percentage = distinct accepted-or-completed evaluators / required reviewers x 100
```

For the prepared example, `2 / 4 x 100 = 50%` of evaluator slots are filled. The separate line below the bar shows how many reviews are completed. Accepted evaluators also occupy slots, so a full bar does not by itself mean every review is complete. The project completes when it reaches its required completed reviewers and has no accepted reviews still ongoing. Status labels and colors help users read the state quickly.

11:00-12:00: show administration briefly. Open review monitoring to show ongoing/completed/overdue information. Then show one evaluator application and its recorded decision in History. Explain the action, affected person, administrator, and date. Use a separate prepared application if recording an approval action.

Suggested narration:

> *The visualization features turn stored review and rating records into readable summaries. Students can follow completion progress, evaluators can understand their feedback, and administrators can monitor review activity.*

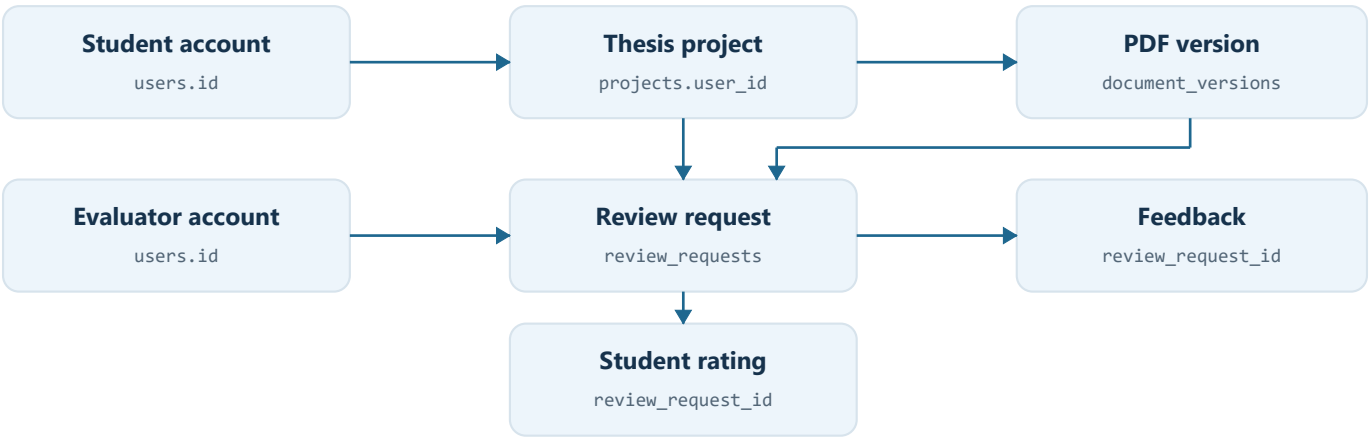
GUI evidence concerns controls, layout, and usability. Visualization evidence concerns what the progress, ratings, and status information communicate. Show the actual data displays in this section.

Handover:

> *These displays come from connected database records. Our final presenter will explain those connections.*

Member 5 - database and closing: relationships

12:00-13:00: show the core relationships. Open the Analista database in phpMyAdmin and use a simple table diagram:



Explain that primary keys identify records and foreign keys connect them. Show only a few relevant columns:

Table	Columns to explain
`users`	`id`, `name`, `role`
`projects`	`id`, `user_id`, `title`
`document_versions`	`id`, `project_id`, `version`
`review_requests`	`id`, `project_id`, `document_version_id`, `evaluator_id`, `status`
`feedback`	`id`, `review_request_id`, `user_id`, `body`
`ratings`	`id`, `review_request_id`, `user_id`, `evaluator_id`, `rating`, `comment`

Member 5 - query and closing

13:00-14:00: show one joined query and its result. Use the same primary project so viewers recognize the title, student, evaluator, PDF version, and completed status.

```
SELECT
    student.name AS student_owner,
    project.title AS project_title,
    evaluator.name AS evaluator_name,
    document.version AS pdf_version,
    request.status AS review_status
FROM review_requests AS request
JOIN projects AS project ON project.id = request.project_id
JOIN users AS student ON student.id = project.user_id
JOIN users AS evaluator ON evaluator.id = request.evaluator_id
JOIN document_versions AS document ON document.id = request.document_version_id
WHERE project.title = 'Campus Appointment and Queue Management System'
ORDER BY request.id DESC;
```

The student column above is the project owner. `review_requests.requested_by` separately records the group member who sent the request. A project with group members uses `project_user` to link those accounts.

14:00-14:30: explain data integrity and file storage. Document versions are unique per project/version; ratings are unique per review/student, so editing a student's rating updates that rating. Explain that PDF files are stored in protected file storage, while the database keeps metadata and references. Avoid displaying passwords, hashes, tokens, `.env`, or private application evidence during the database screen recording.

14:30-15:00: close the presentation.

Suggested narration:

> *Analista combines a usable interface, rule-based review processing, clear progress and rating visualizations, and a connected database. Together, these features organize how students submit thesis work and how volunteer IT evaluators provide and deliver feedback. Thank you for watching our presentation.*

Recording and editing plan

- Record each member's assigned segment separately, using the same project and a consistent screen size. The final video follows the order above.
- Prepare separate browser profiles for student, evaluator, and administrator sessions. Tabs in one browser profile share the same sign-in session.
- Use the title card first, followed by brief lower-third labels containing the member's name, part, and relevant criterion.
- Record the main workflow in its actual order so a pending request, delivered review, and submitted rating exist when later members need them.
- Keep the three role sessions ready before recording. Cut routine logins, repeated typing, loading waits, and mistakes.
- Record readable screen footage, ideally 1920x1080. Zoom in when showing code, a diagram, or SQL results; keep narration at normal speed.
- Give every member audible speaking time. Keep background music low enough for the explanations to be clear.
- Show genuine application behavior and identify separately prepared examples when using them. Rehearse the workflow before recording.
- Export one MP4 and watch the full result once to check text readability, audio levels, transitions, and total duration.

Final evidence checklist

Criterion	The video should visibly demonstrate
GUI - 25%	Navigation, forms, evaluator search/filter, readable cards, role-specific screens
Algorithm - 25%	Eligibility/capacity/conflict decisions, review status transitions, progress and rating calculations
Visualization - 25%	Ratings distribution and average, review progress, meaningful statuses and monitoring
Database - 25%	Core tables and keys, relationships, same-project joined result, document/rating integrity

The main system demonstration should support each explanation. Presentation editing and overlays help the audience follow the system's actual behavior.